

AKADEMIA LEONA KOŹMIŃSKIEGO · INFORMATYKA, I ROK · 24.11.2026

Lab 06

Struktury danych II: słowniki i zbiory

dr inż. Karol Chlasta · WarsawIQ

Repozytorium kursu: github.com/KarolChlasta/KU-PyDev

Cel zajęć

- Rozumieć **tablicę mieszającą** – mechanizm, dzięki któremu słownik i zbiór są szybkie
- Słowniki: tworzenie, metody `get`, `keys`, `values`, `items`
- Zbiory: suma, część wspólna, różnica
- Mapowanie i filtrowanie danych: książka adresowa, duplikaty, szyfr, częstość słów
- Efekty z sylabusa: **U1, U3, KS1**

CZĘŚĆ I

Teoria: mapowania i zbiory

Słownik: klucz → wartość

```
phone = {"Ola": "600100200", "Adam": "500600700"}  
phone["Ewa"] = "700800900"           # dodanie / nadpisanie  
phone["Ola"]                          # '600100200'  
phone["Zenon"]                        # KeyError: 'Zenon'  
phone.get("Zenon")                    # None – bez wyjątku  
phone.get("Zenon", "brak")            # 'brak'  
del phone["Adam"]  
"Ola" in phone                        # True – sprawdza KLUCZE
```

- Klucze są **unikalne**; ponowne przypisanie nadpisuje wartość
- Od Pythona 3.7 słownik **zachowuje kolejność wstawiania** (gwarancja języka)

Metody słownika

WYRAŻENIE	WYNIK
<code>d.keys()</code>	widok kluczy
<code>d.values()</code>	widok wartości
<code>d.items()</code>	widok par <code>(klucz, wartość)</code>
<code>d.get(k, domyślna)</code>	wartość albo domyślna
<code>d.pop(k)</code>	usuwa i zwraca wartość
<code>d.update(inny)</code>	dopisuje / nadpisuje pary
<code>len(d)</code>	liczba par

```
for name, number in sorted(phone.items()):  
    print(f"{name}: {number}")
```

Jak to działa: tablica mieszająca

1. `hash(klucz)` zamienia klucz na liczbę całkowitą
2. Z tej liczby wyliczany jest **numer szufladki** w tablicy
3. Para klucz–wartość trafia do tej szufladki

OPERACJA	LISTA	SŁOWNIK / ZBIÓR
<code>x in ...</code>	$O(n)$ – szuka po kolei	$O(1)$ średnio
dostęp do elementu	po indeksie	po kluczu

- Klucz musi być **hashowalny**: `str`, `int`, `tuple` – tak; `list`, `dict` – nie (są zmienne)
- Od Pythona 3.3 `hash()` tekstów jest **losowany przy każdym uruchomieniu** – ochrona przed atakami DoS, w których napastnik zasypuje serwer kolidującymi kluczami

Zbiory

```
a = {"Ala", "Ola", "Jan"}
b = {"Jan", "Piotr"}
empty = set()           # {} to pusty SŁOWNIK, nie zbiór!
```

OPERACJA	OPERATOR	METODA	WYNIK
suma	<code>a b</code>	<code>a.union(b)</code>	<code>{'Ala', 'Ola', 'Jan', 'Piotr'}</code>
część wspólna	<code>a & b</code>	<code>a.intersection(b)</code>	<code>{'Jan'}</code>
różnica	<code>a - b</code>	<code>a.difference(b)</code>	<code>{'Ala', 'Ola'}</code>
różnica symetryczna	<code>a ^ b</code>	<code>a.symmetric_difference(b)</code>	<code>{'Ala', 'Ola', 'Piotr'}</code>

- Zbiór **nie ma kolejności** ani duplikatów; do wypisania użyj `sorted()`

Wzorzec: zliczanie słownikiem

```
counts = {}  
for word in "to jest to co to jest".split():  
    counts[word] = counts.get(word, 0) + 1  
# {'to': 3, 'jest': 2, 'co': 1}
```

- `get(word, 0)` rozwiązuje problem „pierwszego wystąpienia” bez `if`
- W bibliotece standardowej jest gotowiec: `collections.Counter`

```
from collections import Counter  
Counter("to jest to co to jest".split()).most_common(2)  
# [('to', 3), ('jest', 2)]
```


Którą strukturę wybrać?

POTRZEBUJĘ...	STRUKTURA
kolejności i dostępu po numerze	<code>list</code>
niezmiennego rekordu	<code>tuple</code>
szukania po nazwie / identyfikatorze	<code>dict</code>
unikalności i szybkiego <code>in</code>	<code>set</code>
zliczania wystąpień	<code>dict</code> / <code>Counter</code>

- Dobry wybór struktury danych często ważniejszy niż sprytny algorytm

CZĘŚĆ II

Praktyka

Słownik jako tablica szyfrująca

```
ALPHABET = "abcdefghijklmnopqrstuvwxyz"
encrypt = {}
for i, letter in enumerate(ALPHABET):
    encrypt[letter] = ALPHABET[(i + 3) % 26]
# {'a': 'd', 'b': 'e', ..., 'x': 'a', 'y': 'b', 'z': 'c'}

"".join(encrypt.get(ch, ch) for ch in "ala ma kota") # 'dod pd nrwd'
```

- `% 26` „zawija” alfabet: po `z` wraca `a`
- Słownik odwrotny (deszyfrowanie) zbudujesz, zamieniając klucze z wartościami

Ciekawostki: kryptografia

- Według Swetoniusza **Juliusz Cezar** szyfrował listy, przesuwając litery o 3 – dokładnie jak w `zad3_szyfr.py`
- Szyfr podstawieniowy łąmie się **analizą częstości liter** – opisał ją arabski uczony **al-Kindi** w IX w. To samo liczysz w `zad4_czestosc.py`
- **Marian Rejewski, Jerzy Różycki i Henryk Zygalski** złamali niemiecką Enigmę w latach 30. XX w.; w 1939 r. przekazali wyniki Francuzom i Brytyjczykom

Dzisiejsze zadania

PLIK	ZADANIE
★ <code>zad1_ksiazka_adresowa.py</code>	książka adresowa
★ <code>zad2_duplikaty.py</code>	duplikaty i zbiory
★ <code>zad3_szyfr.py</code>	szyfr Cezara na słowniku
★ <code>zad4_czestosc.py</code>	częstość słów
☆ <code>zad5_znajomi.py</code>	operacje na zbiorach

Sprawdzenie i oddanie

```
cd lab06_slovniki_zbiory
python -m unittest discover -s tests -v
git add .
git commit -m "Lab 06"
git push
```

- Punkt aktywności: **obecność + wszystkie testy ★ zielone + push na GitHub przed 11:45**
-  **Kolokwium I: środa 02.12, 14:00–17:15, A/136** – zakres: laby 01–07

Do poczytania

- E. Matthes, *Python. Instrukcje dla programisty*, Helion 2023 – rozdz. 6
- M. Dawson, *Python dla każdego*, Helion 2015 – rozdz. 5
- L. Ramalho, *Zaawansowany Python*, Promise 2020 – rozdz. 3 (słowniki i zbiory)
- S. Singh, *Księga szyfrów* – historia kryptografii od Cezara do Enigmy

Następny lab (01.12): funkcje + blok pracy z AI 🤖